

Assignment 5 — I2C kommunikasjon mellom to ATmega32

Oskar Hellebust-Nygaard

6. mars 2026

Innhold

1	Innledning	2
2	Kobling	2
3	Teori	3
3.1	I2C-prinsipp	3
3.2	Slaveadresse	4
3.3	TWI-hastighet	4
4	Interrupt-oppsett	5
5	Kodeforklaring	5
5.1	Slave	5
5.2	Master	5
6	i2c.h	6
7	Programkode	6
7.1	Slave-kode	6
7.2	Master-kode	7
7.3	i2c.h	10
8	Resultat	11
9	Feilkilder	12
10	Konklusjon	12

1 Innledning

I denne oppgaven skulle to ATmega32 kobles sammen med I2C (TWI), der en fungerer som slave og en som master.

Begge mikrokontrollerne har:

- 2 LED-er
- 2 knapper

Slaven skal toggle sine egne LED-er med egne knapper.

Masteren skal ved knappetrykk lese statusen til slavens LED-er og kopiere det til sine egne LED-er.

Kravene var:

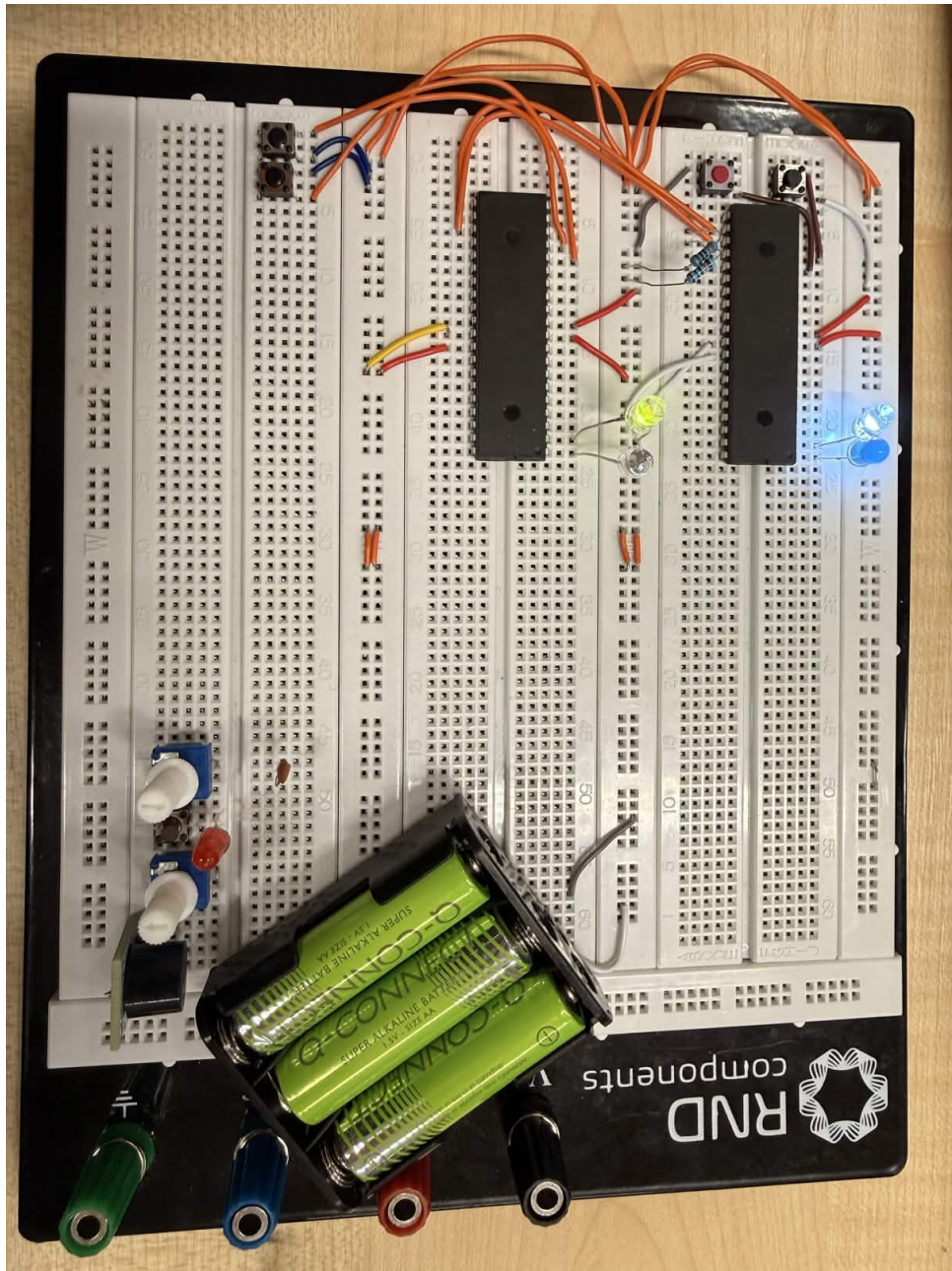
- Interrupt skal brukes
- Event-loop skal være tom
- Egen i2c.h headerfil skal brukes
- Switch-case skal brukes i TWI-statusbehandling
- Alle master-statuskoder unntatt 0x50 skal håndteres

2 Kobling

Begge ATmega32 kobles symmetrisk:

- PB0 og PB1 til LED og GND
- PD2 og PD3 til trykknapper mot GND
- Pull-up brukes internt på knappene
- PC1 = SDA mellom begge kontrollere
- PC0 = SCL mellom begge kontrollere
- Felles GND mellom begge mikrokontrollere
- Felles VCC og AVCC = 5V

I tillegg brukes pull-up på SDA og SCL fordi I2C er open-drain.



Figur 1: Oppkobling av master og slave med I2C

3 Teori

3.1 I2C-prinsipp

I2C bruker to ledninger:

- SDA = data
- SCL = klokke

Master kontrollerer alltid klokkeledningen.
Kommunikasjonen skjer sånn her:

1. START sendes
2. Slaveadresse sendes
3. Read/Write-bit sendes
4. Data sendes eller mottas
5. STOP sendes

3.2 Slaveadresse

Slaven fikk adresse 18:

$$TWAR = (18 \ll 1);$$

Shift brukes fordi adressen ligger i de 7 øverste bitene.

3.3 TWI-hastighet

Master bruker:

$$TWBR = 2;$$

og:

$$TWSR = 0x00;$$

Prescaler = 1.

Bitrate-formel:

$$f_{SCL} = \frac{F_{CPU}}{16 + 2 \cdot TWBR \cdot 4^{TWPS}}$$

Med:

$$F_{CPU} = 1\,MHz$$

får vi:

$$f_{SCL} = \frac{1000000}{16 + 2 \cdot 2} = 50000\,Hz$$

Altså:

$$50\,kHz$$

4 Interrupt-oppsett

Begge knapper bruker falling edge:

```
MCUCR |= (1 << ISC01) | (1 << ISC11);  
MCUCR &= ~((1 << ISC00) | (1 << ISC10));
```

Dette betyr:

- Interrupt trigges når knapp kobles til GND
- Stabilere enn any logical change

5 Kodeforklaring

5.1 Slave

Slave gjør to ting:

1. Knappetrykk toggler LED
2. TWI sender LED-status når master spør

LED-status lagres i:

```
volatile uint8_t LEDStatus = 0x00;
```

Bit 0 = PB0

Bit 1 = PB1

INT0 toggler PB0:

```
PORTB ^= (1 << PB0);  
LEDStatus ^= (1 << 0);
```

INT1 toggler PB1 tilsvarende.

Ved TWI:

- 0xA8 = SLA+R mottatt
- 0xB8 = data sendt, ACK mottatt
- 0xC0 = NACK mottatt

5.2 Master

Master bruker knappetrykk til å starte lesing:

```
sendStart();
```

INT0 betyr kopi av slave PB0.

INT1 betyr kopi av slave PB1.

Når data mottas:

```
slaveData = TWDR;
```

Deretter brukes bit-test:

```
if (slaveData & (1 << 0))
```

for å bestemme om LED skal lyse.

6 i2c.h

Headerfilen samler alle nødvendige TWI-funksjoner:

- sendStart()
- sendSLA()
- sendDATA()
- sendSTOP()
- receiveDATA()
- switchNASM()

Som gjør koden enklere å lese.

7 Programkode

7.1 Slave-kode

```
#define F_CPU 1000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include "i2c.h"

volatile uint8_t LEDStatus = 0x00;

ISR(INT0_vect)
{
    PORTB ^= (1 << PB0);
    LEDStatus ^= (1 << 0);
}

ISR(INT1_vect)
{
    PORTB ^= (1 << PB1);
    LEDStatus ^= (1 << 1);
}

ISR(TWI_vect)
{
    switch (TWSR & 0xF8)
    {
        case 0xA8: //SLA+R received, ACK
            TWDR = LEDStatus; //lagre status i dataregister
            TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE);
            break;

        case 0xB8: //data sent, ACK
```

```

        TWDR = LEDStatus; //hvis master vil ha mer, send
        samme igjen
        TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE);
        break;

        case 0xC0: //data sent, NACK
        case 0xC8: // last data sent
        switchNASM(1); //tilbake til slave-ventemodus
        break;

        default:
        switchNASM(1);
        break;
    }
}

int main(void)
{
    DDRB |= (1 << PB0) | (1 << PB1);

    DDRD &= ~((1 << PD2) | (1 << PD3));
    PORTD |= (1 << PD2) | (1 << PD3);

    TWAR = (18 << 1); //slaveadresse 18
    switchNASM(1); //TWI på, ACK på, interrupt på

    GICR |= (1 << INTO) | (1 << INT1);

    MCUCR |= (1 << ISC01) | (1 << ISC11);
    MCUCR &= ~((1 << ISC00) | (1 << ISC10));

    sei();

    while (1)
    {
    }
}

```

7.2 Master-kode

```

#define F_CPU 1000000UL
#include <avr/io.h>
#include <avr/interrupt.h>
#include "i2c.h"

volatile uint8_t buttonRequest = 0; // 1 = copy LED0, 2 = copy LED1
volatile uint8_t slaveData = 0;

ISR(INT0_vect)
{

```

```

        buttonRequest = 1;
        sendStart();
    }

ISR(INT1_vect)
{
    buttonRequest = 2;
    sendStart();
}

ISR(TWI_vect)
{
    switch (TWSR & 0xF8)
    {
        case 0x08: //START transmitted
        case 0x10: //repeated START transmitted
            sendSLA(18, READ);
            break;

        case 0x40: //SLA+R transmitted, ACK
            receiveDATA(NOT_ACK); //skal bare lese 1 byte
            break;

        case 0x58: //data received, NOT ACK
            slaveData = TWDR;

            switch (buttonRequest)
            {
                case 1:
                    if (slaveData & (1 << 0))
                        PORTB |= (1 << PB0);
                    else
                        PORTB &= ~(1 << PB0);
                    break;

                case 2:
                    if (slaveData & (1 << 1))
                        PORTB |= (1 << PB1);
                    else
                        PORTB &= ~(1 << PB1);
                    break;

                default:
                    break;
            }

            buttonRequest = 0;
            sendSTOP();
            break;

        case 0x18: //SLA+W transmitted, ACK

```



```

        sendSTOP();
        break;

        case 0x20: //SLA+W transmitted, NOT ACK
        sendSTOP();
        break;

        case 0x28: //data transmitted, ACK
        sendSTOP();
        break;

        case 0x30: //data transmitted, NOT ACK
        sendSTOP();
        break;

        case 0x38: //arbitration lost
        sendStart();
        break;

        case 0x48: //SLA+R transmitted, NOT ACK
        sendSTOP();
        break;

        default:
        sendSTOP();
        break;
    }
}

int main(void)
{
    DDRB |= (1 << PB0) | (1 << PB1);

    DDRD &= ~((1 << PD2) | (1 << PD3));
    PORTD |= (1 << PD2) | (1 << PD3);

    TWBR = 2; //enkel TWI-hastighet
    TWSR = 0x00; //prescaler = 1

    GICR |= (1 << INTO) | (1 << INT1);

    TWCR = (1 << TWEN) | (1 << TWIE); //slå på TWI

    //falling edge
    MCUCR |= (1 << ISC01) | (1 << ISC11);
    MCUCR &= ~((1 << ISC00) | (1 << ISC10));

    sei();

    while (1)
    {

```

```
}  
}
```

7.3 i2c.h

```
#ifndef I2C_H_  
#define I2C_H_  
  
#include <avr/io.h>  
  
#define WRITE 0  
#define READ 1  
  
#define ACK 1  
#define NOT_ACK 0  
  
#define LAST 1  
#define NOT_LAST 0  
  
static inline void sendStart(void)  
{  
    TWCR = (1 << TWINT) | (1 << TWSTA) | (1 << TWEN) | (1 <<  
        TWIE);  
}  
  
static inline void sendSLA(uint8_t address, uint8_t read_write)  
{  
    TWDR = (address << 1) | read_write;  
    TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE);  
}  
  
static inline void sendData(uint8_t data, uint8_t last_not_last)  
{  
    TWDR = data;  
  
    if (last_not_last == LAST)  
    {  
        TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE);  
    }  
    else  
    {  
        TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE) |  
            (1 << TWEA);  
    }  
}  
  
static inline void sendSTOP(void)  
{  
    TWCR = (1 << TWINT) | (1 << TWSTO) | (1 << TWEN) | (1 <<  
        TWIE);  
}
```

```

}

static inline void receiveDATA(uint8_t ack_not_ack)
{
    if (ack_not_ack == ACK)
    {
        TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE) |
            (1 << TWEA);
    }
    else
    {
        TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE);
    }
}

static inline void switchNASM(uint8_t recognize)
{
    if (recognize)
    {
        TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE) |
            (1 << TWEA);
    }
    else
    {
        TWCR = (1 << TWINT) | (1 << TWEN) | (1 << TWIE);
    }
}

#endif

```

8 Resultat

Systemet funker slik:

- Slave-knapper toggler slave LED-er
- Master-knapper leser slave-status
- Master PB0 kopierer slave PB0
- Master PB1 kopierer slave PB1

Event-loop er tom:

```

while(1)
{
}

```

All logikk skjer i interrupt.

9 Feilkilder

Feilen som oppstod under testing var:

- PD2 var koblet til VCC i stedet for GND
- Måtte bytte ut flere knapper da de ikke ga reliable kontakt
- Startet med å bruke MISO og MOSI, men innså at det kun brukes for SPI

PD2 til VCC gjorde at knappen og falling edge ikke funket.

10 Konklusjon

Oppgaven oppfyller kravene:

- Interrupt brukes
- Event-loop er tom
- Switch brukes
- I2C fungerer mellom master og slave

Dette ga god forståelse for:

- TWI statuskoder
- Slave/master-logikk
- Registerstyring i ATmega32